

Probability & Token Prediction

How language models assign probabilities and decide what to say next

25 min

Duration

B2 – Understand

Bloom's

L2 – Foundational

Level

M1 · Week 1–2

Module

Learning Objectives

By the end of this session, you will be able to:

B2 – Explain

How an LLM assigns a probability to each token in its vocabulary

B2 – Describe

The next-token prediction task and why it is central to all LLMs

B2 – Interpret

A probability distribution produced by the softmax function

B2 – Distinguish

Between raw logits and normalised probabilities

B3 – Apply

Temperature scaling to shift the output distribution intentionally

B3 – Relate

Token-prediction probabilities to cross-entropy loss and perplexity

Session Roadmap

10 sub-topics in logical dependency order

1. What is a Token?

2. The Next-Token Prediction Task

3. Probability Distribution over Vocabulary

4. Logits — Raw Model Scores

5. The Softmax Function

6. Temperature Scaling

7. Sampling Strategies

8. Cross-Entropy Loss

9. Perplexity

10. Connection to RAG

1.1

What is a Token?

The atomic unit that LLMs see — not words, not characters

What is a Token?

The basic unit of text that a language model processes

Definition

A token is the atomic unit of text that LLMs process — neither a full word nor a single character, but a meaningful subword fragment.

Key Facts

- GPT-4 vocabulary: ~100,000 tokens
- Llama 3 vocabulary: ~128,000 tokens
- 1 token $\approx$ 0.75 English words on average
- Rare words are split into multiple tokens
- Common words are often single tokens

Tokens	Characters
86	345

Definition
A token is the atomic unit of text that LLMs process — neither a full word nor a single character, but a meaningful subword fragment.

Key Facts
GPT-4 vocabulary: ~100,000 tokens
Llama 3 vocabulary: ~128,000 tokens
1 token $\approx$ 0.75 English words on average
Rare words are split into multiple tokens
Common words are often single tokens

Text Token IDs

Example: "Tokenization is fascinating!"

"Token"

"ization"

" is"

" fasc"

"inating"

"!"

Algorithm

Modern LLMs use Byte-Pair Encoding (BPE) — a subword tokenization that balances vocabulary size with coverage of rare words.

1.2

The Next-Token Prediction Task

The single training objective that gives LLMs their capability

The Next-Token Prediction Task

$P(x_{t+1} \mid x_1, x_2, \dots, x_t)$ — the engine of every LLM

Chain Rule of Probability: $P(x_1, x_2, \dots, x_n) = P(x_1) \cdot P(x_2|x_1) \cdot P(x_3|x_1, x_2) \cdot \dots \cdot P(x_n \mid x_1, \dots, x_{n-1})$

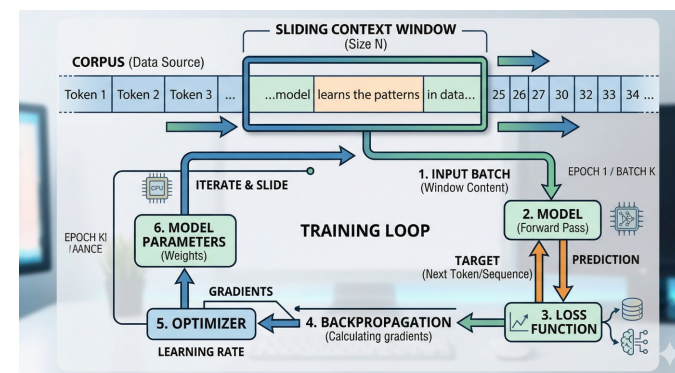


Self-Supervised Learning

No human labels required. The text itself provides supervision — each token is both input and the target for the previous step.

Why One Task Is Enough

Predicting the next word requires grammar, facts, reasoning, and style — all learned implicitly from a single objective.



1.3

Probability Distribution over Vocabulary

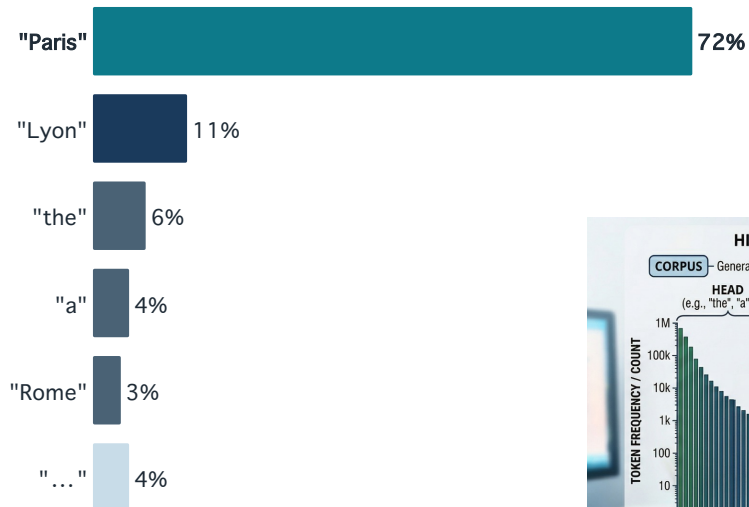
Every forward pass outputs a ranked probability over all possible next tokens

Probability Distribution over the Vocabulary

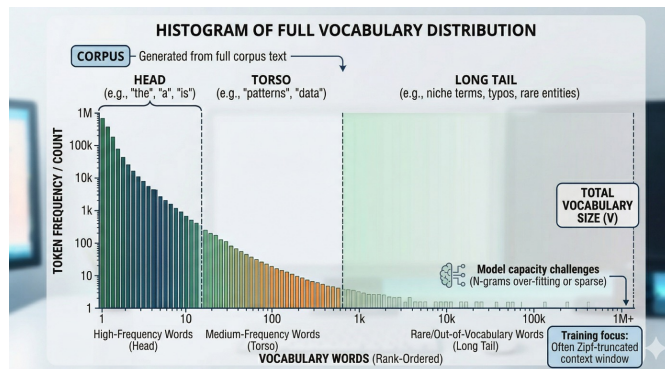
$$p_i = P(x_{t+1} = v_i \mid x_{1:t}), \quad p_i \geq 0, \quad \sum p_i = 1$$

For every forward pass $\rightarrow$ one probability vector $p \in \mathbb{R}^{|V|}$ where each entry ranks a candidate next token

Context: "The capital of France is ____"



- The model NEVER outputs a single word — it always produces a full distribution
- All $|V|$ tokens receive a probability score simultaneously
- High-probability tokens are contextually likely; low-probability are still possible
- Sampling strategy decides which token is ultimately chosen



1.4

Logits — Raw Model Scores

The unnormalised scores before softmax; what the network actually computes

Logits — Raw Unnormalised Scores

$z = W_{lm} \cdot h_t + b$ where h_t is the hidden state at position t

$$z = W_{lm} \cdot h_t + b \quad z \in \mathbb{R} \text{ IVI} \quad (\text{can be any real number})$$

Example logit values (context: 'The capital of France is ____'):

"Paris"	← raw score, not probability	+8.4
"Lyon"	← raw score, not probability	+5.1
"the"	← raw score, not probability	+3.8
"a"	← raw score, not probability	+2.9
"Rome"	← raw score, not probability	+2.4
"cat"	← raw score, not probability	-3.2

Range

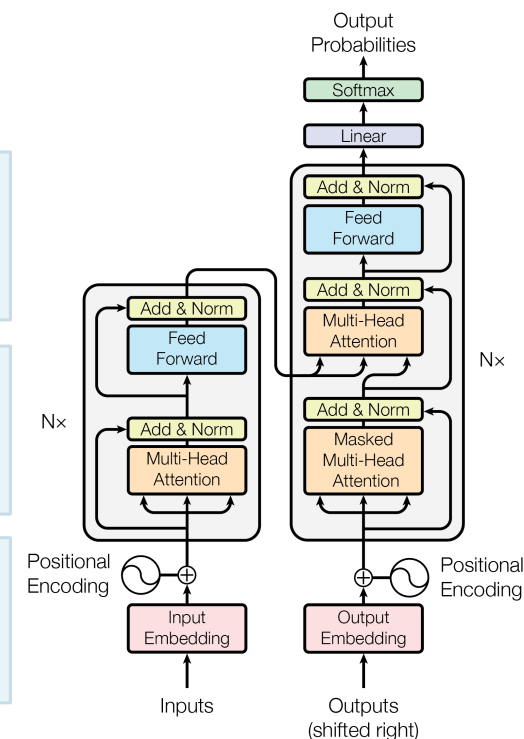
Any real number — positive or negative. Not bounded between 0 and 1.

Higher = preferred

A higher logit means the model 'prefers' that token, but preference $\neq$ probability.

Not yet probabilities

Logits must pass through softmax before they become a valid probability distribution.



1.5

The Softmax Function

Converting raw logits into a valid probability distribution

Softmax: Logits → Probabilities

$p_i = \exp(z_i) / \sum_j \exp(z_j)$ Guarantees: all outputs $\in [0, 1]$, $\sum p_i = 1$

$$p_i = \exp(z_i) / \sum_j \exp(z_j) \quad \text{for all } i \in \{1, \dots, |V|\}$$

Worked Example (4-token mini-vocabulary):

Token	Logit (z)	$\exp(z)$	Probability
Paris	8.4	4447	0.718
Lyon	5.1	164	0.106
the	3.8	45	0.029
Others	...	532	0.147

Sum of $\exp(z)$: $4447+164+45+532 = 5188$ Each probability = $\exp(z) \div 5188$

Why $\exp()$?

$\exp()$ is always positive → ensures no negative probabilities. It also amplifies large logit differences, making confident predictions more decisive.

Why divide by sum?

Normalisation ensures all probabilities sum to exactly 1.0 — making the output a valid probability distribution over the vocabulary.

1.6

Temperature Scaling

One parameter that controls creativity vs determinism

Temperature Scaling

$p_i = \exp(z_i / T) / \sum_j \exp(z_j / T)$ T divides logits BEFORE softmax

$$p_i = \exp(z_i / T) / \sum_j \exp(z_j / T) \quad \text{where } T > 0$$

$T = 0.2$

Low (Deterministic)

Logits divided by small $T \rightarrow$ amplified $\rightarrow$ very peaked distribution

Paris: 98%

Lyon: 2%

Other: 0%

Use case: Factual tasks

$T = 1.0$

Neutral (Default)

Logits unchanged $\rightarrow$ original model distribution

Paris: 72%

Lyon: 11%

Other: 17%

Use case: Balanced

$T = 1.8$

High (Creative)

Logits divided by large $T \rightarrow$ compressed $\rightarrow$ flatter distribution

Paris: 38%

Lyon: 22%

Other: 40%

Use case: Creative writing

1.7

Sampling Strategies

How the model selects one token from the probability distribution

Sampling Strategies

From distribution $\rightarrow$ single token — four approaches compared

Greedy Decoding

$$x = \operatorname{argmax} p$$

Always picks the highest-probability token. Fast, deterministic, but can produce repetitive output.

✓ Reproducible, fast

✗ Boring, repetitive loops

Top-K Sampling

$$p' = \operatorname{renorm}(\text{top-K tokens})$$

Sample only from the K most probable tokens. Others are masked to 0. Controls randomness without full randomness.

✓ Avoids very low-prob tokens

✗ K is fixed, vocabulary-agnostic

Top-P (Nucleus)

$$p' = \text{smallest set where } \sum p \geq P$$

Take the smallest set of tokens whose cumulative probability exceeds threshold P (e.g. 0.9). Adapts to distribution shape.

✓ Context-adaptive, widely used

✗ Slightly harder to tune

Beam Search

keep top-B partial sequences

Maintain B best partial sequences. Used for translation/summarisation where quality > diversity.

✓ Higher quality structured output

✗ Expensive, less creative

1.8

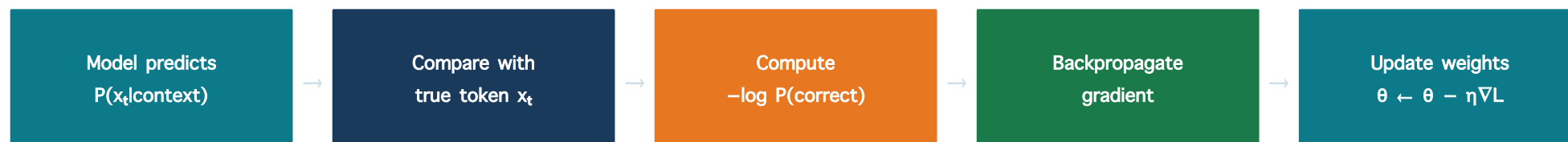
Cross-Entropy Loss

The training signal that teaches the model to assign high probability to correct tokens

Cross-Entropy Loss

$L = -(1/N) \sum_t \log P(x_t | x_1, \dots, x_{t-1})$ averaged over N training tokens

$$L = - (1/N) \sum_t \log P(x_t | x_1, \dots, x_{t-1}; \theta)$$



What loss measures

How much probability the model assigned to the CORRECT next token. If the correct token got $P=0.99$, loss ≈ 0.01 . If it got $P=0.01$, loss ≈ 4.6 . Minimising loss = maximising probability of correct tokens.

Connection to MLE

Cross-entropy minimisation is mathematically equivalent to Maximum Likelihood Estimation — training the model to maximise the probability of the observed training data.

Why $\log P()$?

$\log$ turns multiplication of probabilities into addition (chain rule), and amplifies the penalty for very low probabilities. The $-$ sign flips it so we minimise instead of maximise.

1.9

Perplexity

The standard metric for evaluating how well a language model fits text

Perplexity — Evaluation Metric

$$PPL = \exp(L) = \exp(-(1/N) \sum_t \log P(x_t | \text{context}))$$

PPL = exp(L) Lower PPL = Better model = Higher probability assigned to correct tokens

PPL benchmarks on WikiText-103 (lower = better):

GPT-2 (1.5B)	29.4	2019 baseline
GPT-3 (175B)	20.5	Major improvement
Llama 2 (70B)	8.7	Open-weight SOTA
GPT-4	~6–7	Current frontier
Human text	~2–4	Upper bound

Intuition

PPL = "effective vocabulary size the model is choosing from". PPL=8 means on average the model is deciding between 8 equally likely tokens — much better than random choice over 100,000 tokens.

Limitations

PPL measures fit to training distribution — not factual accuracy, coherence, or helpfulness. A model can have low PPL and still hallucinate.

1.10

Connection to RAG

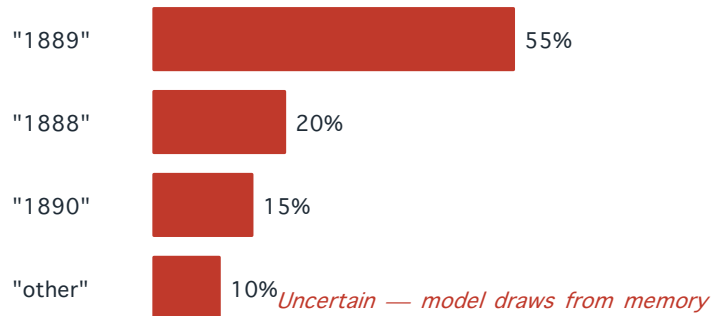
How retrieved context shifts the probability distribution — and why it matters

How RAG Shifts the Probability Distribution

No weight changes — only the context changes. But probabilities shift dramatically.

Without RAG

Query: "When was the Eiffel Tower built?"

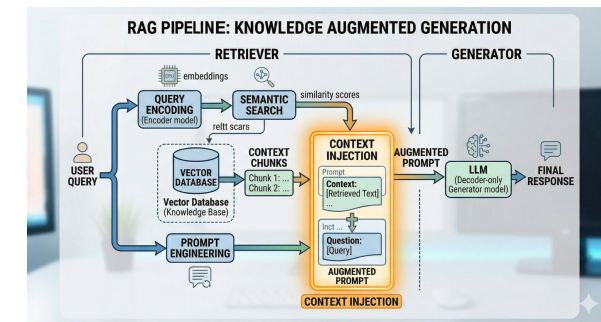


With RAG

Context: "...Eiffel Tower completed 1889..." + Query



Key Insight: RAG does NOT change model weights. It changes the context → context changes the logits → logits change the softmax → probabilities shift toward factually correct tokens.



Assignment A1 — Probability & Token Prediction

One week to complete · Mentors discuss submissions in the following session

WHAT · WHY · EXPECTED OUTCOME

WHAT

Call the API with logprobs=True. Retrieve the top-5 token probabilities at the first predicted position. Verify they sum to 1.0.

Re-run the same prompt at $T = 0$, $T = 0.7$, and $T = 1.5$. Record the top-5 distribution at each setting in a comparison table.

WHY

To observe softmax behavior directly — not as a formula on a slide, but as real numbers that change with a parameter you have already used.

Connects the mathematical definitions from today's lecture to the API parameter you configured in Module 2.

EXPECTED OUTCOME

A table showing how probability mass shifts across the three temperature settings.

Three sentences in your own words explaining what temperature is actually controlling.

If you can write that explanation correctly, you have understood the softmax.

Deliverable: Jupyter notebook — logprobs output, 3-temperature comparison table, written explanation · Submit before mentor session

Common Misconceptions to Clear Up

Correct understanding before we move to self-attention

X High probability = factually correct

- ✓ High confidence means the model is consistent with its training distribution, NOT that it is correct. Hallucinations can occur at $P > 0.95$.

X The model picks the highest probability word

- ✓ Only greedy decoding does that. Most inference uses top-p or top-k sampling — the model intentionally introduces randomness.

X Temperature changes the model

- ✓ Temperature only scales logits at inference time. Model weights are unchanged. Two calls at different temperatures use identical weights.

X Lower perplexity always means better model

- ✓ PPL measures fit to a specific text distribution. A model can have low PPL and still hallucinate freely on facts outside its training data.

X Softmax is specific to language models

- ✓ Softmax is the standard output for any multi-class classification network. In LLMs it just runs over 100,000+ classes instead of 10.

X Tokens are words

- ✓ "tokenization" = 3 tokens. "cat" = 1 token. "ChatGPT" may be 2–3 tokens. Tokenization depends on frequency in the training corpus.

References & Further Reading

Foundational papers, tools, and resources for Topic 1

Foundational Papers

- [1] Shannon, C.E. (1948). A Mathematical Theory of Communication.
Origin of entropy, the basis for cross-entropy loss
- [2] Vaswani et al. (2017). Attention Is All You Need. NeurIPS.
Transformer architecture — softmax drives attention too
- [3] Kaplan et al. (2020). Scaling Laws for Neural Language Models.
How perplexity scales with model size and data
- [4] Brown et al. (2020). Language Models are Few-Shot Learners. (GPT-3)
Demonstrates what next-token prediction alone can produce at scale
- [5] Holtzman et al. (2020). The Curious Case of Neural Text Degeneration.
Nucleus (top-p) sampling paper — motivation and method

Tools & Videos

- Visualiser Andrej Karpathy — Let's build GPT from scratch (YouTube)
youtube.com/watch?v=kCc8FmEb1nY
- Visualiser The Illustrated GPT-2 — Jay Alammar's blog post
jalamar.github.io/illustrated-gpt2
- Tool OpenAI Tokenizer — tiktoken playground
platform.openai.com/tokenizer
- Tool Hugging Face logprobs — transformers output_scores=True
huggingface.co/docs
- Course 3Blue1Brown — But what is a neural network? (YouTube)
3blue1brown.com
- Book Jurafsky & Martin — Speech & Language Processing Ch. 3
web.stanford.edu/~jurafsky/slp3

Topic 1 Summary — Probability & Token Prediction

The six essential things to take away

1

Token ≠ Word

LLMs operate on subword tokens. GPT-4 has ~100K tokens. Tokenization determines sequence length and cost.

2

One task, all capability

Next-token prediction via chain rule is the only training objective. Grammar, facts, and reasoning all emerge from it.

3

Softmax = logits → probs

Raw logits pass through $\exp()$ / $\sum \exp()$ to become a valid probability distribution. All IVI tokens are ranked simultaneously.

4

Temperature controls sharpness

$T < 1 \rightarrow$ confident, deterministic. $T > 1 \rightarrow$ flat, creative. T divides logits BEFORE softmax — it does not change weights.

5

Loss = how much probability was on the right token

$L = -(1/N) \sum \log P(\text{correct token})$. Minimising loss = maximising probability of observed training tokens.

6

RAG shifts context, not weights

Retrieved documents change what the model sees $\rightarrow$ logits shift $\rightarrow$ probabilities shift toward factually grounded tokens.

Topic 1 Complete

Probability & Token Prediction

Coming up next in this session:

Topic 2

Autoregressive Generation

The generate loop: predict → append → repeat. Live API demo with streaming.

Topic 3

Self-Attention Mechanism

Q, K, V — the library analogy, formula derivation, causal masking.

Topic 4

Multimodal AI & Vision Transformers

Images as token sequences. ViT patch decomposition. M4 Section 1.

Assignment A1 issued · Submit before mentor session next week · Questions during the break